Appendix: PatchGuru: Patch Oracle Inference from Natural Language Artifacts

Anonymous Authors

I. Prompts

A. Oracle Inference Prompt

The following listing shows the oracle inference prompt used by PatchGuru to infer developer intent and generate a test driver with Python assertions comparing pre-PR and post-PR function behaviors.

Listing 1: Oracle inference prompt.

```
# Role
You are an expert software developer. Your task is to:
- Infer the developer's intent behind the pull request (PR).
- Write Python assertions comparing the pre-PR and post-PR versions.
- Generate a test driver that can be executed to validate the assertions automatically.

# Guidelines

1. **Understand the Context:** This is your starting point. Read the original implementations of the function(s) before the PR and the broader context in which they operate, such as the enclosing class or module, if applicable. Understand what the function(s) are supposed to do, their inputs, outputs, and any edge cases they handle. This foundational knowledge is crucial for effective review.

2.  **Infer the Developer's Intent:** Next, read the PR details carefully. Look for clues in the title, description, and any comments, discussions and related references (e.g., PRs and issues). Then try to answer the following questions:
- What is the developer trying to accomplish?
- Is it a bug fix, a new feature, or a performance tweak?
- What specific problem or limitation in the original code is being addressed?
- How does the proposed change improve or alter the original semantics?
Your entire review hinges on your accurate understanding of this intent.

3.  **Define the Expected Behavior:** Based on the inferred intent, clearly state the desired outcome. For example, if it's a bug fix, describe what the system **should do now** that it didn't do before. If it's a new feature, describe the exact functionality it must provide.

4.  **Identify the Invariant Properties:** What aspects of the system should remain unchanged after the PR is merged? For example, if a database query is being optimized, the data it returns should be the same. This helps ensure that the change doesn't introduce unintended side effects.

5.  **Create a Testable Hypothesis:** This step is crucial for validating code changes in PR. In ideal cases, your hypothesis should not just describe the old and new behaviors in isolation, but instead state the explicit causal relationship between them. It is the narrative that your assertions will prove.

    - **Ideal Format:** "Given input `X`, the pre-PR version returns `Y`, while the post-PR version will return `Z`, where `Z` is a filtered version of `Y`."

    - **Example:**

        - **Inferred Intent:** A fix for a user profile data sanitization function. The bug is that it correctly filters profanity but fails to remove malicious URLs. The fix should add URL filtering.
        - **Hypothesis:** "When a user-provided string containing both profanity and a malicious URL is passed to the sanitization function, the pre-PR version will return a string with the profanity removed but the URL intact, while the post-PR version will return a string with both the profanity and the URL removed."
```

````
6.  **Write Precise Python Assertions:** This is a key step. You will write assertions that directly express the relationship between the pre-PR and post-PR versions. Instead of just asserting the final outcome, your assertions will compare the behaviors of both versions. Ensure that your assertions are precise, unambiguous, and directly tied to the inferred intent and hypothesis. Besides, you MUST provide generalized assertions that can be executed various input values. Avoid hardcoding specific input values in your assertions.

    - **Example (Data Filtering from above):**
      - **Scenario Setup:** A mock string containing both types of unsanitized data.
      ```python
      def pre_sanitize_input(input_string):
          # ... returns the sanitized string
          pass

      def post_sanitize_input(input_string):
          # ... returns the sanitized string
          pass

      malicious_input = "This is a great product, but visit: http://malicious-site.com"

      old_output = sanitize_input_pre_pr(malicious_input)
      assert "malicious-site.com" in old_output, "The pre-PR version should have kept the malicious URL."

      new_output = sanitize_input_post_pr(malicious_input)
      assert "malicious-site.com" not in new_output, "The post-PR version should have removed the URL."
      assert new_output == old_output.replace("http://malicious-site.com", ""), "The post-PR output should be the pre-PR output with the URL removed."
      ```
NOTE THAT, you can disregard any assertions or checks regarding the content of stdout, stderr, or logging.

7. **Construct a Test Driver.** Create a test driver that can be run to automatically validate your assertions. This driver should be runnable in an isolated environment where both the **pre-PR** and **post-PR** versions of the function(s) are accessible. Your test driver must include:
    - A function contains assertions.
    - A `main` function to orchestrate the testing process.
    - Clear placeholders for a reviewer to add the concrete implementations of the **pre-PR** and **post-PR** functions.

    - **Important:**
      - **Directly** call the function(s) being tested. Do not use objects, libraries, or other indirect methods.
      - The `main` function **must** define independent input variables for testing the function(s) in isolation.
      - Provide a set of test cases for these input variables to ensure your assertions are executed meaningfully.

8.  **Document and Explain:** In your review, present your findings clearly and concisely. Start by stating the inferred intent, provide the test driver code, and then explain exactly what the assertions prove.

# Input

You will be provided the pull request details, including the PR title, description, and conversations between developers, and the source code of the function(s) before the PR, as follows:

## Pull Request Details

{pr_details}

## Enclosing Class of target function(s) (If Applicable)

{context}
````

```
## Source code of target function(s) before the pull request

```python
{code_of_pre_pr_function}
```

Function signatures of target function(s) after the pull request

```python
{signature_of_post_pr_function}
```

Available Imports
You can also refer to the available imports and assume they are already imported in the test driver:
```python
{available_import}
```

Output Format Instructions

1. Assertions for different behaviors should be denoted clearly in their error messages by corresponding tags [CHANGED BEHAVIORS], [NEW BEHAVIORS], [PRESERVED BEHAVIORS]
- [CHANGED BEHAVIORS] describe the behaviors of the function(s) that were changed by the PR.
- [NEW BEHAVIORS] describe the behaviors of the function(s) that were added by the PR.
- [PRESERVED BEHAVIORS] describe invariant properties that should be preserved by the PR.

2. Function names of the pre-PR and post-PR versions should be denoted by "pre_" and "post_" prefixes, respectively. For example, if the function name is "calculate_sum", the pre-PR version should be named "pre_calculate_sum" and the post-PR version should be named "post_calculate_sum".

3. Test drivers should be carefully documented in the comments to describe how test cases are generated and what behaviors they are testing.

4. MUST NOT provide concrete implementation of the function(s) {prev_fut_names} in the test driver. These concrete implementations will be added later in the placeholders in the section "# Source Code of target function(s)".

5. DON'T USE pytest or unittest libraries in the test drivers. Instead, use Python's built-in assert statements to create assertions.

6. Make sure to include main function to execute the test drivers.

7. Please refers to available imports when writing the test drivers. Do not include import statements that are already provided in the available imports.

8. Finally, provide your response in the following format:

<reasoning>
[Put your reasoning chain here including the analysis of the PR details and the source code of the function(s) before the PR.]
</reasoning>

<hypothesis>
[Put your testable hypothesis here that describes the relationship between the pre-PR and post-PR versions of the function(s).]
</hypothesis>

<test_driver>
Neccessary Imports

Source Code of target function(s)
Before Pull Request
[Placeholders for pre-PR function, e.g., def pre_function(): ...]

After Pull Request
[Placeholders for post-PR function, e.g., def post_function(): ...]

```

```
Specification
[Put Python assertions comparing the pre-PR and post-PR versions of the function(s) here, along with a main function to execute the assertions.]

</test_driver>
```

B. Oracle Enhancement Prompt

The following listing shows the oracle enhancement prompt used by PatchGuru to generate bug-triggering test cases from a given specification.

Listing 2: Oracle enhancement prompt.

```
# Role
You are an expert software developer. You will be provided an specification written in form of Python assertions that describes the relationship between the pre-PR and post-PR versions of the function(s). Your task is to generate more potential input test cases for the assertions in the specification. These test cases should be designed to trigger potential bugs in the post-PR version of the function(s). Your goal is to create test cases that can effectively validate the correctness of the post-PR implementation.

# Guidelines

1. **Understand the Context:** This is your starting point. Read the implementations of the function(s) before and after the PR and the broader context in which they operate, such as the enclosing class or module, if applicable. Understand their inputs, outputs, and side effects what the function(s) are supposed to do in two versions. This foundational knowledge is crucial for effective review.

2.  **Understand the Developer's Intent:** Next, read the PR details and specification written in Python assertions. Then try to answer the following questions:
- What is the developer trying to accomplish?
- Is it a bug fix, a new feature, or a performance tweak?
- What specific problem or limitation in the original code is being addressed?
- How does the proposed change improve or alter the original semantics?
This understanding will help you focus your testing on the most relevant aspects of the change.

3.  **Generalize Assertions:** Thoroughly examine the provided **Python assertions**. If an assertion is specific (e.g., `assert post_pr(5) == pre_pr(5) + 1`), immediately identify and formulate the **general principle** it represents (e.g., "The post-PR function should always return the pre-PR result plus one, *for all valid inputs x*"). Then, rewrite the assertion in a more generalized form (e.g., `assert post_pr(x) == pre_pr(x) + 1 for all valid x`). This generalization is crucial because it allows you to think beyond the specific example and consider the broader behavior of the function. Your tests must validate this general rule, not just the specific example. Note that, you should only modify the assertions to make them more general, but you MUST NOT change their fundamental meaning or intent. Also, if behavior changes are introduced in specific input domains, DON'T over-generalize those assertions.

4.  **Identify Edge Cases and Boundary Conditions:** Test the limits of the function's expected inputs. Some common edge cases include but are not limited to:
    - **Extremely large/small** values (e.g., maximum/minimum integers, floats).
    - **Minimum/Maximum** values (e.g., 0, 1, the largest/smallest integer the system supports).
    - **Boundary** conditions (e.g., inputs that switch behavior, like N and N+1).
    - **Empty/Null** inputs (e.g., empty lists, empty strings, None).
    - **Single-element** inputs (for sequences).

5.  **Leverage Domain Knowledge for "Gotchas":**
    Think like a developer implementing the *post-PR* code. Based on the likely nature of the change (the PR), generate tests for common pitfalls in that domain, such as:
    - **Off-by-one errors** (e.g., i < N vs. i <= N).
    - **Concurrency/State** issues (if the function is stateful).
    - **Floating-point precision** issues (e.g., testing 0.1 + 0.2 vs. 0.3).
    - **Modulo/Division by Zero** conditions.
    - **Type conversions** (e.g., int to float, string to int).
    - **Data structure mutations** (e.g., modifying a list while iterating over it).

```

````
6.  **Focus on Differences Between Pre- and Post-PR:**
    Hypothesize *why* the code was changed and *where* the new logic might break. Design tests that specifically target the **new logic or modified paths** in the post-PR version. The goal is to trigger the **bug** the developer might have introduced while making the requested change.

7.  **Generate Non-Trivial, Interacting Inputs:**
    Move beyond simple, standalone inputs. For functions that accept multiple arguments, create tests where the **combination** of arguments reveals a bug, or where complex data structures interact in unexpected ways.

8.  **Prioritize Input Diversity and Distribution:**
    Ensure your generated inputs cover a wide range of the function's domain. Don't just test small integers; test large ones, very large ones, negative ones, and ones close to specific constants like 2^k. Use techniques like **randomized testing** guided by the input constraints, and then curate the most suspicious outputs. You should aim to generate around 10-15 diverse test cases that cover different aspects of the function's input space.

9.  **Ensure Test Case Independence and Clarity:**
    Each generated test case should ideally test one specific concept or bug trigger. Clearly document the **intent** of each generated test (e.g., "Test for off-by-one error at maximum allowed size," or "Test for type conversion failure with float input").


# Input

You will be provided specification written in Python assertions that describes the relationship between the pre-PR and post-PR versions of the function(s), along with pull request details, including the PR title, description, and conversations between developers, the source code of the function(s) before and after PR, as follows:

## Specification

{specification}

## Pull Request Details

{pr_details}

## Enclosing Class of target function(s) (If Applicable)

{context}

## Source code of target function(s) before the pull request

```python
{code_of_pre_pr_function}
```

## Source code of target function(s) after the pull request

```python
{code_of_post_pr_function}
```

## Available Imports
You can also refer to the available imports and assume they are already imported in the test driver:
```python
{available_import}
```

# Output Format Instructions

1. Please keep existing information in the specification intact. You should only add more test cases to the specification and modify existing assertions to make them more general, but you MUST NOT change their fundamental meaning or intent.
````

```
2. Function names of the pre-PR and post-PR versions should be denoted by "pre_" and "post_" prefixes, respectively. For example, if the function name is "calculate_sum", the pre-PR version should be named "pre_calculate_sum" and the post-PR version should be named "post_calculate_sum".

3. Test drivers should be carefully documented in the comments to describe how test cases are generated and what behaviors they are testing.

4. MUST NOT provide concrete implementation of the function(s) {prev_fut_names} in the test driver. These concrete implementations will be added later in the placeholders in the section "# Source Code of target function(s)".

5. DON'T USE pytest or unittest libraries in the test drivers. Instead, use Python's built-in assert statements to create assertions.

6. Make sure to include main function to execute the test drivers.

7. Please refers to available imports when writing the test drivers. Do not include import statements that are already provided in the available imports.

8. Finally, provide your response in the following format:

<reasoning>
[Put your reasoning chain here including the analysis of the PR details and the source code of the function(s) before the PR.]
</reasoning>

<hypothesis>
[Put your testable hypothesis here that describes the relationship between the pre-PR and post-PR versions of the function(s).]
</hypothesis>

<test_driver>
# Neccessary Imports

# Source Code of target function(s)
## Before Pull Request
### [Placeholders for pre-PR function, e.g., def pre_function(): ...]

## After Pull Request
### [Placeholders for post-PR function, e.g., def post_function(): ...]

# Specification
## [Put your generalized specification with added test cases here.]

</test_driver>
```

C. Self-Review Prompt

The following listing shows the self-review prompt used by PatchGuru to determine whether a failing test driver indicates a real bug in the PR or a flawed test.

Listing 3: Self-review prompt.

```
# Role
You are an expert software developer. Your task is to analyze a test driver that is generated by an AI system for testing a pull request (PR) in a codebase. The test driver is failing with assertion errors when run against the new code introduced by the PR. Your goal is to determine whether the failure indicates a real bug in the PR or a flawed test driver that does not accurately reflect the developer's intent.

# Guidelines

Your task is to act as a detective, distinguishing between a real bug and a flawed test. Here are guidelines to help you analyze failing test drivers against a new PR.

1.  **Read the PR Details Carefully:** Start with the developer's own words. The PR description and any linked issues/PRs are your first source of truth. Look for clues about
```

```
   what the developer intended the code to do. What problem does the PR aim to solve? Are
   there any known limitations or edge cases mentioned?

2. **Read Code Changes Carefully:** Examine the code changes introduced by the PR. Understand
    what the new code is supposed to do, how it interacts with existing code, and any new
   assumptions it introduces. Pay attention to test cases added or modified in the PR, as
   they often reflect the developer's intent.

3.  **Examine the Test's Intent:** Read the failing test itself. What is it trying to prove?
   Pay close attention to the test's name, comments, and the specific assertion that is
   failing. Is the test checking for a specific value, a particular data type, or a
   behavioral change?

4.  **Cross-Reference the Test Driver with PR Details:** Compare the failing test with the
   expected behavior as described in the PR. Does the test align with the developer's stated
   intent? If the test expects behavior that contradicts the PR description, that's a strong
   indicator of a mismatch.

5.  **Analyze the Failure Data:** Look at the actual versus expected values in the assertion
   error. The difference between what the test expected and what the code produced is the
   most concrete piece of evidence you have. Sometimes the difference is a small detail - a
   changed data type, a rounding error, or a slightly different output format. This is often
   where a mismatch in intent becomes clear.

6.  **Identify Documentation Gaps:** If the PR's behavior deviates from what the tests assume
   , but you believe the new behavior is correct and intentional, the problem might be
   insufficient documentation. A good PR should clearly explain any breaking changes or new
   assumptions. If it doesn't, that's a potential bug in the PR's submission, not the code
   itself.

7.  **Consider Unintended Side Effects:** The bug might not be in the feature the PR is
   directly addressing. The changes could have a knock-on effect on another part of the
   codebase that the failing test is checking. A test failure in an unrelated area is a
   strong indicator of a real, unintended bug.

8.  **Look for Alternative Interpretations:** If the PR description is vague, consider
   alternative ways the code could be interpreted. Could the developer have a different
   understanding of a term or a function's purpose? This requires empathy and a willingness
   to see the problem from their perspective.

9. **Formulate a Clear Conclusion:** Based on your analysis, state your findings clearly. Is
   it a **bug** (the code is not doing what it should) or a **mismatch** (the test is out of
   date or based on incorrect assumptions)? Provide evidence from your analysis and recommend
    a clear path forward, such as fixing the code, updating the test, or clarifying the PR's
   documentation.

10. **Suggest Corrections if Needed:** If you identify a mismatch, suggest how the test
   driver could be updated to better reflect the intended behavior of the code. You might
   recommend changes to the test's assertions, input values, or even its overall purpose. You
    MUST NOT suggest changes to the PR code itself. If you find a bug, you should not do
   anything further than identifying it, as fixing the bug is outside the scope of your role.

# Input

You will be provided the test driver, the error message from the failed test, pull request
   details, including the PR title, description, and conversations between developers, and
   code changes introduced by the PR, as follows:

## Pull Request Details

{pr_details}

## Code Changes Introduced by the Pull Request

'''
{code_changes}
'''

## Test Driver
```

```
```python
{test_driver}
```

Error Message

{error_message}

Output Format Instructions

Provide your response in the following format:

<reasoning>
[Put your reasoning chain here including the analysis of the pull request details, the function code before the PR, and the test driver. This should include your thought process on how you arrived at your conclusion.]
</reasoning>

<conclusion>
[State your conclusion here: "BUG" if the failure indicates a real bug in the PR, or "MISMATCH" if the failure is due to a flawed test. Only put "BUG" or "MISMATCH" here in uppercase letters.]
</conclusion>

<test_driver>
[Put the corrected test driver here if you concluded "MISMATCH". Make sure that fixed code is structurally similar to the input code. The fixed code should still contain three main sections: "# Neccessary imports", "# Specification", and "# Source Code of target function(s)". You can assume that the "# Source Code of target function(s)" section is unchanged and do not need to provide it again in your response. But you should still include the section header "# Source Code of target function(s)" between # Neccessary imports and # Specification sections in your response. If you concluded "BUG", you MUST leave this section empty.]
</test_driver>
```

D. Error Repair Prompts

PatchGuru uses three specialized prompts to repair errors in generated test drivers.

1) Reasoning Error Repair Prompt: The following listing shows the prompt used to repair assertion errors caused by incorrect specifications when executed against the pre-PR version.

Listing 4: Reasoning error repair prompt.

```
# Role
You are an expert Python software engineer. Your task is to correct assertion errors provided a test driver that validates the expected relationship between two versions of a function: before and after a pull request. Currently, the test driver is facing an AssertionError in the pre-PR version, indicating wrong specifications. Your task is to fix the specifications so that the test driver runs without assertion errors in the pre-PR version.

# Guidelines

To successfully fix the test driver and resolve the **AssertionError** in the pre-PR version, here are up to five key guidelines you should follow:

1. **Analyze the Test Driver and Function Code:** First, carefully examine the test driver's code and the pre-PR version of the function. Understand what the test is trying to validate and how the function is supposed to behave. The `AssertionError` is a symptom, not the root cause. You need to understand the underlying logic of both the test and the function to find the mismatch.

2. **Identify the Incorrect Specification:** The `AssertionError` tells you exactly where the test driver's expectation is not being met. Pinpoint the specific assertion that's failing. The message provided with the error (if any) will likely point to the line number and a description of what failed, which is your most valuable clue. The "specification" you need to correct is the expected output, input, or relationship defined in the assertion itself.
```

```
11
12 3.  **Adjust the Test Driver, Not the Function:** Your role is to fix the *specifications* in
       the test driver. **Do not modify the pre-PR function code.** The goal is to make the test
       accurately reflect the function's correct behavior *before* the pull request. Changing
      the function would defeat the purpose of validating the PR's changes. The fix lies in
      adjusting the `assert` statement's parameters to match the function's actual, correct
      output.
13
14 # Input
15
16 You will be provided a test driver that is used to validate the expected relationship between
       two versions of a function: before and after a pull request. The code is expected to run
      without errors. However, it is currently faced with ASSERTION errors as follows:
17
18 ## Test Driver
19
20 ```python
21 {code}
22 ```
23
24 ## Error Message
25
26 ```
27 {error_message}
28 ```
29
30 # Output Format Instructions
31
32 1. Make sure that fixed code is structurally similar to the input code. The fixed code should
       still contain three main sections: "# Neccessary imports", "# Specification", and "#
      Source Code of target function(s)".
33
34 2. You can assume that the "# Source Code of target function(s)" section is unchanged and do
      not need to provide it again in your response. But you should still include the section
      header "# Source Code of target function(s)" between # Neccessary imports and #
      Specification sections in your response.
35
36 3. Provide your response in the following format:
37
38 <reasoning>
39 [Put your reasoning chain here including the analysis of the error message and the test
      driver code. This should include your thought process on how you arrived at the fix.]
40 </reasoning>
41
42 <fixed_code>
43 [Put the fixed code here that resolves the assertion error. Ensure that the code is
      executable and maintains the original semantics of the test driver.]
44 </fixed_code>
```

2) Runtime Error Repair Prompt: The following listing shows the prompt used to repair runtime errors in generated test drivers, such as `TypeError`, `NameError`, `IndexError`, or `ValueError`.

Listing 5: Runtime error repair prompt.

```
1 # Role
2 You are an expert Python software engineer. Your task is to correct runtime errors in the
      provided Python code based on the accompanying error message.
3
4 # Guidelines
5
6 1. **Only Debug on Target Sections**: Provided Python code is a test driver with three main
      sections: "# Neccessary imports", "# Specification", and "# Source Code of target function
      (s)". You MUST focus all debugging efforts on the "# Neccessary imports" and "#
      Specification" sections. The "# Source Code of target function(s)" section is guaranteed
      to be correct and should not be changed.
7
8 2.  **Analyze the Full Traceback:** Runtime errors usually provide a full traceback that
      shows the sequence of function calls leading to the error. Start by reading the error
      message at the bottom to understand the type of error (e.g., `TypeError`, `ValueError`, `
```

```
    KeyError`). Then, work your way up the traceback to see the exact line of code where the
    error occurred and the function calls that led to it.

3.  **Check for Common Runtime Errors:** Be on the lookout for frequent issues:
    - **`TypeError`:** This often happens when you perform an operation on an object of the
      wrong data type (e.g., trying to add a string to an integer).
    - **`NameError`:** The code is trying to use a variable or function that has not been
      defined or is out of scope.
    - **`IndexError` or `KeyError`:** You are trying to access an item in a list or
      dictionary with an index or key that does not exist.
    - **`ValueError`:** A function receives an argument of the correct type but an
      inappropriate value (e.g., `int("abc")`).

# Input

You will be provided a test driver that is used to validate the expected relationship between
    two versions of a function: before and after a pull request. The code is expected to run
    without errors. However, it is currently faced with RUNTIME errors as follows:

## Test Driver

```python
{code}
```

Error Message

```
{error_message}
```

Output Format Instructions

1. Make sure that fixed code is structurally similar to the input code. The fixed code should
    still contain three main sections: "# Neccessary imports", "# Specification", and "#
    Source Code of target function(s)".

2. You can assume that the "# Source Code of target function(s)" section is unchanged and do
    not need to provide it again in your response. But you should still include the section
    header "# Source Code of target function(s)" between # Neccessary imports and #
    Specification sections in your response.

3. Provide your response in the following format:

<reasoning>
[Put your reasoning chain here including the analysis of the error message and the test
    driver code. This should include your thought process on how you arrived at the fix.]
</reasoning>

<fixed_code>
[Put the fixed code here that resolves the runtime error. Ensure that the code is executable
    and maintains the original semantics of the test driver.]
</fixed_code>
```

3) Syntax Error Repair Prompt: The following listing shows the prompt used to repair syntax errors in generated test drivers, such as missing brackets, incorrect indentation, or missing imports.

Listing 6: Syntax error repair prompt.

```
# Role
You are an expert Python software engineer. Your task is to correct syntax errors in the
    provided Python code based on the accompanying error message.

# Guidelines

1. **Only Debug on Target Sections**: Provided Python code is a test driver with three main
    sections: "# Neccessary imports", "# Specification", and "# Source Code of target function
    (s)". You MUST focus all debugging efforts on the "# Neccessary imports" and "#
    Specification" sections. The "# Source Code of target function(s)" section is guaranteed
    to be correct and should not be changed.
```

````
7
8  2.  **Read the Error Message Carefully:** The traceback is your most valuable tool. Python's
       error messages are often very specific and will tell you the exact file, line number, and
       type of error. Pay close attention to the error type (e.g., `SyntaxError`, `
       IndentationError`, `NameError`) and the message that follows, as it often hints at what is
        wrong.
9
10 3.  **Focus on the Indicated Line and its Neighbors:** Once you identify the line number from
        the error message, examine that line of code first. If the error isn't immediately
       obvious, look at the line directly above it. Syntax errors often occur because of a
       missing colon, parenthesis, or bracket on the preceding line.
11
12 4.  **Check for Common Syntax Mistakes:** Be on the lookout for frequent errors such as:
13     - **Missing or Mismatched Brackets/Parentheses:** Ensure every opening of bracket,
           parenthesis, or brace has a corresponding closing one.
14     - **Incorrect Indentation:** Python uses indentation to define code blocks. Mixing spaces
            and tabs or having an inconsistent number of spaces will cause an `IndentationError`.
15     - **Missing Dependencies:** Ensure all necessary imports are present and correctly
           spelled.
16
17 # Input
18
19 You will be provided a test driver that is used to validate the expected relationship between
        two versions of a function: before and after a pull request. The code is expected to run
       without errors. However, it is currently faced with SYNTAX errors as follows:
20
21 ## Test Driver
22
23 ```python
24 {code}
25 ```
26
27 ## Error Message
28
29 ```
30 {error_message}
31 ```
32
33 # Output Format Instructions
34
35 1. Make sure that fixed code is structurally similar to the input code. The fixed code should
        still contain three main sections: "# Neccessary imports", "# Specification", and "#
       Source Code of target function(s)".
36
37 2. You can assume that the "# Source Code of target function(s)" section is unchanged and do
       not need to provide it again in your response. But you should still include the section
       header "# Source Code of target function(s)" between # Neccessary imports and #
       Specification sections in your response.
38
39 3. Provide your response in the following format:
40
41 <reasoning>
42 [Put your reasoning chain here including the analysis of the error message and the test
       driver code. This should include your thought process on how you arrived at the fix.]
43 </reasoning>
44
45 <fixed_code>
46 [Put the fixed code here that resolves the syntax error. Ensure that the code is executable
       and maintains the original semantics of the test driver.]
47 </fixed_code>
````

II. Detected Bugs

The full list of real-world bugs detected during evaluation is summarized below.

TABLE I: Real-world bugs detected by PatchGuru.

Repo	ID	PR	Description	Kind	Status
Pandas	1	60828	Intends to fix a bug in `_pprint_seq` that displayed a frozen set in "()" rather than "{}"; the fix was incorrect and the bug remains.	Code bug	Fixed independently
	2	61054	Intends to fix a bug in `DataFrame.loc[]` that returned incorrect data types, but did not cover slice inputs	Code bug	Confirmed and fixed (merged)
	3	61162	NL states that it enables `Series[bool]` indexing for `iloc.__getitem__` without specifying scope; however, applies only to integer-indexed series.	Doc bug	_
	4	61183	Intends to fix a regression in `Series.interpolate` but coincidentally handled `NaN` values that previously raised `NotImplementedError`.	Doc bug	_
	5	61646	Intends to fix a bug in `Series.var` that produce incorrect results for complex arrays but do not address the root cause and the bug persists.	Code bug	Confirmed and fixed (merged)
	6	61946	Intends to fix a bug in `str.contains` that raised a `TypeError` with compiled regular expressions, but introduced incorrect results.	Code bug	Fixed independently
	7	61966	Intends to fix the representation of string categories, but applies only to `str` and does not address `string`.	Code bug	Confirmed and fixed (merged)
	8	62248	Intends to fix a bug in `core.frame` that did not preserve column names when `nrows == 0` but overlooked edge cases with excluded columns.	Code bug	Confirmed and fixed (merged)
SciPy	9	22213	Intends to support complex-valued `k` in `scipy.signal.zpk2tf` but return wrong results with multi-dim `z`	Code bug	Confirmed and fixed (merged)
	10	22475	Intends to fix a bug in `optimize.shgo` which fails with `jac = True`; however, this fix cause the function to modify user supplied dictionaries.	Code bug	Fixed independently
	11	22532	Intends to avoid warnings for `stats.genextreme`, but caused a regression in `stats.genextreme._stats`; the regression does not propagate to the public API.	Doc bug	_
	12	22989	Intends to improve efficiency of `hierarchy.is_isomorphic`; coincidentally fixes a bug that caused the function to accidentally accept string inputs.	Doc bug	_
	13	23280	Intends to fix broadcasting to allow array-valued a and b but coincidentally fix bug on calculation.	Doc bug	_
	14	23341	Intends to support `array-api-strict` in `signal.zpk2tf` with multi-dim `z` but introduce a regression on non-integer `k`	Code bug	Confirmed and fixed (merged)
	15	23520	Intends to fix a bug in `optimize.shgo` which mutates user supplied dictionaries but overlook the edge case of nested dictionaries	Code bug	Confirmed
Marshmallow	16	1399	Intends to refactor code but coincidentally fix a bug that provides wrong args for class-based objects	Doc bug	_
	17	2698	Intends to change formatting specifiers in `get_fixed_timezone` from `%` to `.format()` but cause regression	Code bug	Fixed independently
	18	2699	Intends to fix a bug in `get_fixed_timezone` that returned an incorrect string representation of time	Code bug	Removed in latest version
	19	2800	Intends to handle `file` URLs, but works only with lowercase schemes and ignores URLs with uppercase letters.	Code bug	Confirmed and fixed (merged)
Keras	20	20626	Intends to fix layer normalization with scalar mean and variance, but broke it for multidimensional cases.	Code bug	Confirmed and fixed (merged)
	21	20765	Intends to fix a bug in `TimeDistributed` layer with partial batch size but introduce a regression that silently accepts mask with mismatched timesteps.	Code bug	Confirmed and fixed (merged)
	22	20879	Intends to solve a bug in `linalg.solve` with `tensorflow` backend, but breaks compatibility with `tf.function` with unknown size. May be acceptable, but needs to be clearly documented.	Doc bug	_
	23	20928	Intends to implement `arctan2` for the `OpenVINO` backend; however, contains a bug that makes `arctan2` return non-NaN values when given NaN inputs.	Code bug	Confirmed and fixed (merged)
	24	20974	Intends to refactor `image.gaussian_blur` for `numpy` efficiency, but broke it on even-sized images.	Code bug	Confirmed and fixed (merged)